SLAM: An SLA-aware, Cost-efficient Blockchain Framework

Abstract

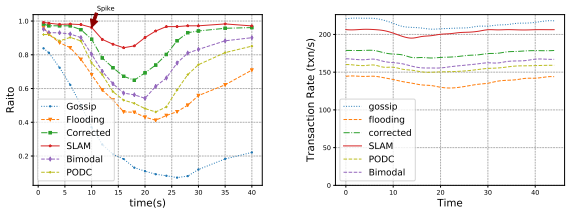
While the success of the blockchain paradigm has attracted more and more smart contract applications to be deployed on a blockchain system consisting of a P2P network, no service-level agreements (SLA) guarantee has been achieved for enforcing the deadlines of smart contract transactions. The clients which generate these transactions often invoke these applications interactively (e.g., via web browsers and mobile phones), so it is highly desirable to enforce a stringent transaction commit latency (e.g., tens of seconds) as SLA to ensure reasonable client experience. The stateful nature of a smart contract often requires each client's transactions to be committed and executed sequentially (no gap exists in the sequence). Existing P2P reliable multicast protocols designed for enforcing the complete sequence all conduct heavyweight gap-filling and refreshing (fetching latest transactions from peers), making these protocols unsuitable for meeting SLA deadlines for a blockchain's P2P network. Moreover, specific protocol messages (e.g., gap-filling) in this protocol are vulnerable to selective targeted attacks in a blockchain deployment.

This paper presents SLAM, the first SLA-aware and reliable multicast protocol designed to enforce SLA deadlines for transactions securely and efficiently in a blockchain. SLAM leverages the strong security features of Intel SGX to develop a message-oblivious P2P reliable multicast protocol to defend against selective targeted attacks; it also leverages the block commit feature to make its own protocol lightweight and fast on enforcing per-client sequence of smart contract transactions. Evaluation on the Ethereum blockchain system with three state-of-the-art P2P reliable multicast protocols and five diverse real-world applications that desire SLA requirements in transactions shows that: (1) SLAM completely eliminates targeted attacks on its SLA-enforcing messages; and (2) even with the existence of transaction spikes and attacked nodes, SLAM achieves much high SLA satisfaction rate for transactions than all the three relevant reliable multicast protocols with a reasonable transaction throughput. Our SLAM source code and evaluation results are released on github.com/osdi20p264.

1 Introduction

The emergence of blockchains has attracted the developments of diverse Internet-wide data sharing applications (e.g., e-voting [6, 16], disease control [1, 2], and online payment [9]) on permissioned blockchain systems (e.g., Ethereum [32], Hyperledger Fabric [5], Quorum [8]). While a blockchain system can greatly improve the reliability of these applications, including eliminating a central authority and improving accountability, these applications still naturally desire two important requirements as their past deployments in a traditional distributed database. The first requirement is the service-level agreements (SLA) on sequential execution: for transactions generated by each client, if some transactions invoke smart contracts [11], these transactions must be committed onto the blockchain with the same complete order (i.e., no gap exists) as the order generated by the client. The second requirement is the SLA on commit latency: many of these transactions are generated by each client on web browsers or mobile phones interactively, so the commit latency of these transactions should not exceed tens of seconds. In this paper, we call the transactions that desire these two requirements as "SLA transactions".

Unfortunately, despite much efforts on developing these Internet-wide applications on fast blockchain systems (e.g., Ethereum-PoA [32]), fulfilling these two important requirements for Internet-wide applications is still especially challenging. For reliability and network bandwidth efficiency, existing blockchain systems' P2P networks, which handle the disseminations of transactions submitted by clients, often adopt a probability-based Gossip protocol [21] to conduct the disseminations. If a blockchain's consensus nodes, which run a consensus protocol to determine which block to commit, find that a smart contract transaction is lost during the probabilistic but non-full dissemination, the client which generated this transaction is often required to resubmit the transaction repeatedly. This can easily violate the SLA re-



(a) SLA guarantees of differnet systems (b) Throughput of different systems

Figure 1: SLA guarantee and performance under spike

quirement of this transaction and those of all the client's subsequent transactions.

Ethereum allows an application to specify the option of a smart contract's execution deadline, and some applications' deadlines can be short (e.g., a blockchain-driven online action [**?**] can end in one minute). However, none of Ethereum, other blockchain systems, or research work has presented a guaranteed mechanism to enforce this deadline when the transactions are being disseminated throughout a blockchain's P2P network. Specifically, existing blockchain systems' P2P multicast protocols not only often miss deadlines, but also often incur gaps in per-client transaction sequence on consensus nodes (§7).

Intuitively, a blockchain system's ideal multicast protocol that aims to satisfy these two important SLA requirements should embrace two key components. First, each P2P node should prioritize the dissemination of SLA-stringent transactions over non-SLA transactions. Second, to prevent clients' retrying and redisseminating the transactions throughout the P2P network, the protocol's multicast should incorporate state-of-the-art reliable P2P multicast mechanisms [**?**, 10, 18, 25], so that P2P nodes can exchange received transactions with their neighbors and redisseminate only the lost transactions. In fact, Erlay [25], a latest work, has integrated a P2P reliable multicast protocol [10] with Bitcoin [24] to improve both the bandwidth efficiency of Bitcoin's P2P network and privacy of clients.

However, integrating existing P2P reliable multicast protocols [**?**, 10, 18, 25] with a blockchain system still fall short in meeting the two important requirements of sequential execution and SLA for Internet-wide applications. The reason is that existing reliable multicast protocols are mainly designed for MPI [14], so they have to achieve both the gap-filling (SLA on sequential execution) and refreshing (all or certain nodes receive all latest transactions) tasks with high probability. The refreshing task is especially costly in terms of network bandwidth, CPU usage and time for all nodes in the P2P network, because each node needs to start from itself, uni-directionally and recursively inquiries their peers to infer the latest transactions throughout the network, a kind of flooding multicast [24].

Leveraging the blocks' commit-then-disseminate nature in a blockchain, we come up with a simple and lightweight P2P reliable multicast protocol: during the dissemination of client transactions (the forward directional dissemination), our protocol need only conduct lightweight and efficient gap-filling of client transactions and guarantee that the consensus nodes of a blockchain can receive each client's SLA transactions without gaps. On the perspective of a consensus node, this guarantee is especially desirable because consensus can now efficiently commit the per-client gap-free transactions in a block and efficiently disseminate this committed block throughout the P2P network (the backward directional dissemination). This backward directional dissemination can also help the P2P nodes between a client and a consensus node to safely skip the gap-filling tasks of the committed transactions. Overall, our simple bi-directional reliable multicast protocol can greatly reduce the commit latency in the perspective of clients.

Our SLAM system presented in this paper is based on this idea. However, making this new bi-directional protocol practical in the blockchain domain for SLA transactions requires tackling two major technical challenges. First, we must create a decentralized SLA prioritization mechanism to make P2P nodes disseminate SLA-stringent transactions with high priority. Existing P2P reliable multicast protocols [**?**, 10, 18, 25] lack such a prioritization mechanism. They just disseminate all packets equally regardless of their elapsed dissemination time cost, so the packets with smaller remaining elapsed time (if this time is set) are often be greatly deferred by packets with the larger time even in an uncongested P2P network (confirmed in our evaluation).

SLAM's core is an SLA-aware and Reliable Multicast (SLARM) protocol for a blockchain's P2P network. To tackle this challenge, SLAM's protocol includes a new decentralized SLA prioritization mechanism, which lets each P2P node update the remaining SLA deadline of each SLA transaction conservatively according to the transaction's elapsed time during the cross peer dissemination. We provide a theoretical proof (§4.3) and evaluation (§7) to how that SLAM's mechanism is robust to tough (e.g., congested) network scenarios.

The second challenge is about security, which partially stems from SLAM's solution for the first challenge. Specifically, to judge whether a transaction is SLA-stringent, each SLA transaction in SLAM is piggybacked with the SLA metadata such as the remaining time to its deadline. The SLA metadata is updated by each node during transmission. In a blockchain deployment, some nodes can be faulty and corrupt the metadata. Moreover, specific protocol messages (e.g., gap-filling) in SLAM's multicast protocol are vulnerable to selective targeted attacks.

Fortunately, the increasingly prevalent trusted execution hardware (e.g., Intel SGX [22]) has strong confidentiality and integrity features to tackle the second challenge against

faulty nodes. With SGX, SLAM's protocol enables two important security features. The first feature is integrity: in SLAM's decentralized transaction SLA dissemination mechanism, each P2P node only accesses the metadata within the node's SGX. Second, to handle targeted attacks on specific messages (e.g., gap-filling), SLAM's protocol is designed to be oblivious among all P2P messages, including client transaction dissemination messages and all SLAM's protocol messages.

We implemented SLAM base on Ethereum [32] and evaluated SLAM on both our cluster and AWS [**?**]. We compared SLAM with three state-of-the-art P2P reliable multicast protocols [**?**, 10, 18, 25], and with two traditional P2P multicast protocols (Gossip [**?**] and flooding [3]). Specifically, Erlay [25], base on bimodal multicast [10], is a latest reliable multicast protocol designed for client transaction dissemination in a blockchain. We evaluated all these protocols and SLAM with three blockchain consensus protocols and five real-world applications with diverse portions of SLA transactions, non-SLA transactions, and transaction spikes. Our evaluation and analysis show that:

1. SLAM completely eliminates targeted attacks on its SLA-enforcing messages (§5.3);
2. Figure 1a shows that even with the existence of transaction spikes, SLAM achieves much higher SLA satisfaction rate for SLA transactions than all the three relevant reliable multicast protocols and the two traditional multicast protocols;
3. Figure 1 shows that, compared to Gossip, SLAM's bi-directional reliable multicast achieves a reasonable overhead of 19.X% on the throughput of all transactions (including both SLA and non-SLA transactions).
4. SLAM is robust on DoS attacks against nodes.

The major novelty of this paper is SLAM, the first P2P reliable multicast protocol that can meet the two important SLA requirements in the blockchain domain. SLAM's protocol is secure, efficient, and can support both general consensus protocols and applications. SLAM's integration with Ethereum has the potential to attract broad Internet-wide applications with SLA requirements to be developed upon. SLAM can enable the research community to develop even more interesting blockchain systems and applications with heterogeneous SLA requirements (§7.5).

The remaining TODO

2 Background

2.1 Permissioned Blockchain

A blockchain runs a distributed consensus protocol among a group of nodes as consensus nodes to agree on which block can be committed in a total order onto the blockchain, where a block contains a large number of transactions submitted by many clients. A transaction is defined as committed if a block containing this transaction is committed. This paper mainly involves the Ethereum blockchain system [32]; its P2P network implementation is the most mature among open-source blockchain systems. Ethereum can be either deployed in a permissionless or a permissioned way. This paper focuses on a permissioned blockchain, which requires all nodes joining its network to go through an explicit membership registration process [5, 32], due to two main reasons. First, a permissioned blockchain often has much higher throughput (Ethereum-PoA [**?**] commits about 200 txns/s) and lower latency than a permissionless blockchain (e.g., Bitcoin [**?**] commits about 7 txs/s), so a permissioned blockchain faces a more stressful P2P network. Second, a permissionless blockchain has a cryptocurrency incentive scheme to motivate P2P nodes to disseminate a more valuable transaction (a kind of SLA scheme), where a permissioned blockchain lacks such a scheme.

It is essential for clients in a permissioned blockchain to submit transactions via P2P disseminations throughout the network in order to reach consensus nodes. In a permissioned blockchain's P2P network, all P2P nodes are granted members of the permissioned blockchain. Some P2P nodes can be elected as consensus nodes to agree on committing which block onto the blockchain. Some consensus protocols (e.g., PoA [**?**]) periodically re-elect consensus nodes among all nodes in the P2P network, while some other protocols (e.g., Algorand [15]) make their consensus nodes be hidden from other nodes and clients. Therefore, this P2P dissemination of client transactions throughout the P2P network is essential.

The life cycle of a client transaction mainly contains two parts: (1) the dissemination of the transaction from the client throughout the P2P network, and (2) the dissemination of a committed block containing this transaction. The second part often takes a minor portion of the time cost of the entire life cycle, because many transactions are packed in one committed block (i.e., in batches, for many clients, and gap-free) to disseminate, and each client typically connects to multiple nearby P2P nodes to fetch the committed block. As long as one of the connected nodes receives the block, the client infers the transaction as committed. The second part also often happens less frequently (due to a block's batched nature) and consumes much less network bandwidth than the first part. The first part is the major time bottleneck and improvement spot of the life cycle (§7), because many clients disseminate their transactions throughout the P2P network with the existence of ad-hoc network contention and transaction gaps, and this part is the only part that can be improved in the perspective of an individual client's perceived transaction commit latency. Our evaluation shows that the second part was often stable and took about 5.7s (only 33.5% of the entire life cy-

cle) in the PoA (Proof-of-Authority [**?**]) consensus protocol.

2.2 Intel SGX

Intel Software Guard eXtension (SGX) [22] is a popular and prevalent trusted execution hardware product. SGX provides a secure execution environment called enclave. Data and code in an enclave can not be tampered with or revealed from outside. A process running outside the enclave can invoke an SGX `ECall` to switch its execution into the enclave and to execute a statically-shielded function in the enclave; a process running in an enclave can invoke an `OCall` to switch its execution outside the enclave.

3 Overview

3.1 Failure and Threat Model

Same as typical permissioned blockchains' P2P networks, we consider SLAM's network an asynchronous, Internet-wide network. For nodes that have joined SLAM's permissioned blockchain (network), any components of these nodes running outside of SGX can be faulty. Any SLAM node with a faulty component is called a faulty node in this paper. Specifically, faulty nodes can randomly drop and delay clients' transactions as well as SLAM protocol messages. Faulty nodes can also try to alter the SLA metadata in SLAM's SLA-aware multicast protocol, and can selectively detect SLAM's gap-filling messages from all received messages and delay or drop them. All nodes have only loosely synchronous clocks, and faulty nodes can manipulate their local clocks. Due to the asynchronous nature of Internet, node failures and dropping transactions or messages can not be distinguished by other nodes, so SLAM treats a failed node as faulty. A client's transactions are signed by the client itself and sent to connected peers (§2.1), so faulty nodes can not modify the transactions or their SLA requirements.

To prevent faulty nodes propagating an incorrect SLA deadline metadata to its peers (e.g., assigning a stringent deadline to an SLA transaction with a loose deadline), SLAM's runtime system updates only the metadata within each node's SGX enclave launched by SLAM. Outside the SLAM's enclaves, all client transactions (including both SLA and non-SLA transactions) are encrypted to make all transactions oblivious. In each node's enclave, all transactions are decrypted (§4.1). In SLAM, all the firmware and hardware components of SGX are trusted. Hiding traffic endpoints (Karaoke [19] and Stadium [28]), SGX micro-architecture side-channels [31], and SGX Iago attacks [13] are out of the scope of this paper.

3.2 SLAs in SLAM

An SLA is an agreement [**?**] between a customer and a service provider on the service quality of diverse requirements (e.g., performance and reliability). In this paper, our SLAM system is the service provider, and an application and its clients are the customers. Driven by the emergence of the blockchain paradigm, more and more applications are deployed in blockchains using smart contracts. A smart contract [**?**] is a stateful deterministic program stored on the blockchain that can be invoked via client transactions submitted to the blockchain. Many of these applications have two SLA requirements. First, a client's transactions that invoke smart contracts must be sequentially executed (gap-free) according to the order submitted by the client. Therefore, if a gap exists in a client's transactions received by consensus nodes, all the client's transactions after the gap can not be committed. The second requirement is SLA on commit latency: many of these transactions are generated by a client on interactive web browsers or mobile phones, so the commit latency (life cycle) of these transactions should not exceed tens of seconds.

To meet these two SLA requirements, in SLAM, the SLA of an application's transaction is a 2-tuple $sla{:}\{Deadline, Order\}$, where *Deadline* is the desired commit latency of the transaction, and *Order* is whether the transaction needs to be executed in order. Suppose there is no network congestion, and suppose the median commit latency of a transaction is T (e.g., 16.8s in the Ethereum-PoA system with Gossip in Table 3). Given an application (e.g., online payment in Table 2) deployed on SLAM, SLAM by default considers all smart contract transactions in this application as SLA transactions, and by default sets the two tuples as $(c \times T, Yes)$ for all the SLA transactions, where c is a configurable constant among clients. Because all clients and P2P nodes have loosely synchronous clock in SLAM, and faulty nodes can manipulate their local clock, SLAM uses the latency duration instead of absolute clock time.

In SLAM, suppose in every second there are n SLA transactions with deadline $c \times T$ being submitted to the blockchain network. If n does not exceed the maximum throughput of the consensus protocol deployed in SLAM and the network capacity of SLAM's P2P network, then, SLAM guarantees that all SLA transactions can meet their SLAs with high probability p (96.0% when $c = 2$, proved in §4.3). Note that, given the same conditions, all typical blockchains' Gossip [**?**], flooding [**?**], and existing reliable multicast protocols [**?**, 10, 18, 25] enforce much lower SLA satisfaction rate than SLAM (Figure 1a) when they are integrated into blockchains (e.g., Erlay [25]).

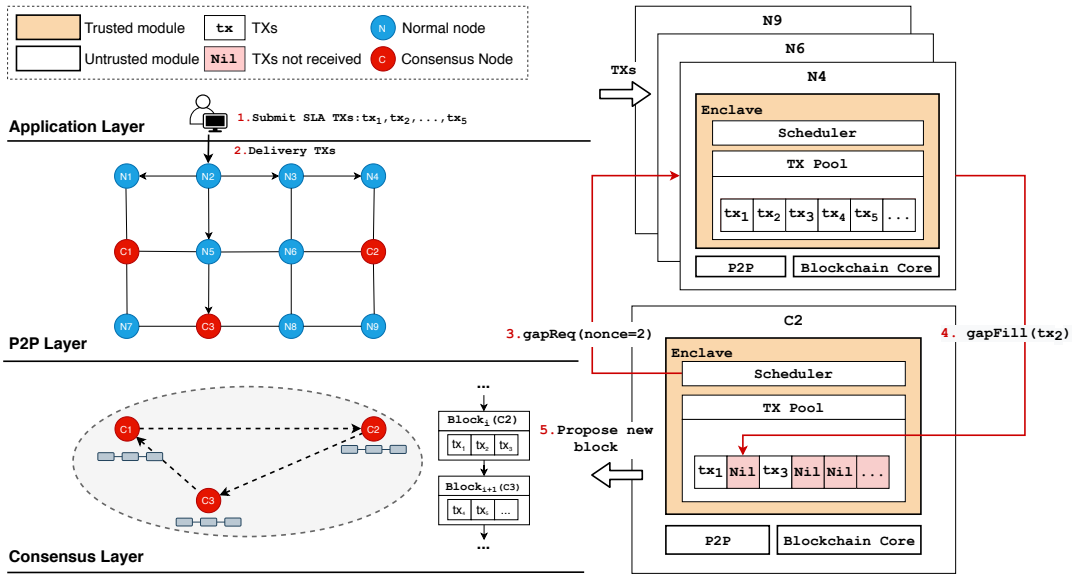


Figure 2: SLAM's architecture

3.3 SLAM Architecture

Figure 2 shows SLAM's architecture. Each SLAM node, including both normal nodes and consensus nodes (§2.1), has two trusted modules and two untrusted modules. The scheduler and reliable multicast module running in an SGX enclave (orange color) are trusted. On a normal node, the scheduler module prioritizes client transactions sent from the node's peers according to the transactions' SLAs and forwards them to the multicast module on the same node. On a consensus node, the scheduler module does the same prioritization work and forward these transactions to the blockchain core module on the same node. On both a normal node and a consensus node, the multicast module conducts a gap-filling task when it finds a gap from its received per-client SLA transaction sequence; for non-SLA transactions, gap-filling is unnecessary. Then, the reliable multicast module continues the transaction dissemination according to the priority decided by the scheduler module. If this node is a consensus node, the complete sequence of transactions is forwarded to the blockchain core module.

An SLAM enclave launched on a SLAM node consumes little memory, because these two trusted modules are 1.7k LOC (§6), and the enclave memory stores only uncommitted transactions. On each SLAM node, the blockchain core module (including the consensus protocol) and network communication module (including TCP/UDP) are outside SLAM's enclave and are not trusted. The blockchain core module performs consensus on committing which block, maintains a local copy of the blockchain, and executes committed transactions extracted from committed blocks. The communication module conducts peer discovery/adjustment and disseminates transactions forwarded from the reliable multicast module to the node's peers.

Figure 2 also shows the life cycle of an SLA transaction in SLAM: (1) A client submits a series of SLA transactions via a nearby blockchain node N2. (2) After N2's scheduler module receives these transactions, it prioritizes the propagation of them according to their SLAs (§4.1). The reliable multicast module propagates transactions to a random subset of peers via a standard Gossip protocol [**?**] using a `disseminate` message. (3) If a node C2 detects a gap (tx_2) in received transactions, C2 sends a `gapFill` message to request for tx_2 from a random subset of its peers (§4.2). (4) If a node in the subset has the requested transaction, the node replies C2 with a `gapRly` message. In SLAM, non-SLA transactions are disseminated using SLAM's default Gossip protocol without gap-filling. (5) The consensus nodes pack transactions into blocks and agree on which block to commit next.

To make SLAM support practical blockchain applications with SLA transactions, we must tackle a selective targeted attack in P2P networks: recent work [**?**] shows that attackers can selectively defer certain types of P2P protocol messages during a client transaction's dissemination, which can trigger the default peer adjustment mechanism in a P2P network and maliciously make certain victim nodes adjust most of their peers to faulty nodes. Such an Eclipse attack can further arbitrarily delay the dissemination of transactions of these victim nodes and violate the transactions' SLA in SLAM. Moreover, even if such Eclipse attacks do not exist, because the protocol behavior and sizes of `gapFill` messages and normal `disseminate` messages are explicitly distinguishable in existing P2P reliable multicast protocols (see Figure 4), attackers can easily stop the gap-filling tasks by dropping or

deferring their messages and stop the entire commit progress of many clients' SLA transactions. Existing reliable multicast protocols [**?**,10,18,25], including Erlay [25], a latest notable P2P reliable multicast protocol tailored for blockchain security, are especially vulnerable to such attacks. §4 and §5 will present how SLAM tackles such attacks and enforces SLA with high probability.

4 The SLAM Basic Protocol

This section first describes SLAM's decentralized SLA prioritization mechanism, which updates each SLA transaction's remaining SLA deadline on each hop node when this transaction is being disseminated throughout the network. By doing so, nodes will give more SLA-stringent transactions higher priorities during disseminations. On a SLAM node N, when disseminating an SLA transaction, this mechanism subtracts the deadline of this transaction with two trustworthy (secure) time variables: $N.HRTT$, the worst case (conservative) Half Round Trip Time cost node N's peers sending a transaction to N; tx_N^{wait}, the time cost an SLA transaction tx spent on node N's scheduler queue. §5 will present how SLAM makes this two variables *conservative*: SLAM can avoid faulty nodes' malicious behaviors of making the subtracted elapsed time from a transaction's remaining SLA deadline smaller than the actual elapsed time of the transaction's dissemination path, including selective deferring attacks and manipulating local clocks. To ease discussion, this section assumes these two variables are conservative first.

4.1 Prioritizing SLA Transactions

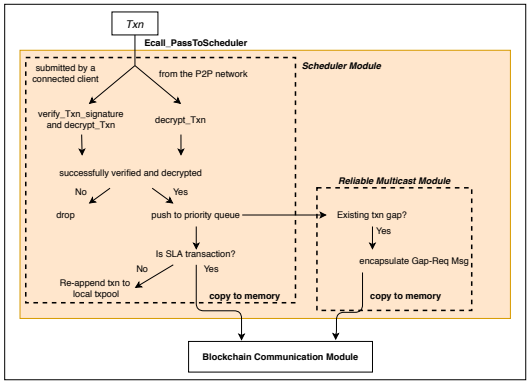


Figure 3: SLAM's enclave code invocations (ECall) and outbound p2p messages (sent via Ocalls). Enclave code are shaded orange.

We first present SLAM's SLA prioritization mechanism in a high level. Figure 3 shows that, when a SLAM P2P node N receives a transaction tx sent from a client, n submits the transaction to its local SLAM enclave by a `ECall`. The scheduler module running in n's SLAM enclave (§3.3) verifies tx's signature and retrieves tx's SLA requirements $c \times T, Yes$ (if any). The scheduler then generates an SLA metadata m which indicates the transaction's initial remaining SLA deadline ($c \times T$), encrypts the transaction tx body and its metadata m together using a symmetric key shared among all nodes' SGX enclaves launched by SLAM (§6), and disseminates the encrypted transaction to a random subset of peers using Ethereum's Gossip protocol (§4.2). Upon receiving a transaction from a peer, a SLAM node decrypts the transaction in SLAM's local enclave and schedules the transaction according to its remaining deadline in m. Before sending the transaction to other peers, node N's multicast module (§3.3) updates and re-encrypts the transaction's SLA metadata.

In a detailed level, for each scheduler module of a SLAM node, this module runs within an SGX enclave (Figure 3) to prioritize SLA transactions over non-SLA transactions. This module is essential because typical applications often have a large portion of non-SLA transactions (Table 2). Without this module, non-SLA transactions can easily block SLA transactions and violate SLA (§7). Moreover, transactions with the same SLA requirement can have different remaining SLA deadlines in SLAM.

In the perspective of a client, the end-to-end commit latency of a transaction contains two parts: the client transaction dissemination latency τ_d (from a client to consensus nodes) and the consensus latency τ_c (including the time cost of agreeing which block to commit and that of the committed block reaching a connected peer of the client, see §2.1). According to our evaluation (§7.1), tau_d takes up about 52.3% - 86.1% of the end-to-end commit latency and varies significantly with the transaction workload. The consensus latency (τ_c) is often stable (§2.1) and the block commit that causes this latency is infrequent (e.g., happens once in the network every 5s in PoA [**?**]). Therefore, τ_c does not involve SLAM's scheduler module, and SLAM just subtracts τ_c from a transaction's SLA deadline once when a transaction is sent from its client to a connected P2P node. SLAM focuses on making τ_d meet the remaining SLA deadline.

To meet SLA transactions' deadlines in SLAM's P2P network, on each SLAM node, SLAM's scheduler module gives SLA-Stringent transactions higher priority to be transmitted on the node's multicast module. Specifically, when the scheduler module receives incoming transactions (Figure 3), it decrypt all the transactions' SLA metadata m. If a transaction is an SLA transaction, the scheduler module subtracts the remaining deadline with the actual time spent in transferring tx from N's peer (suppose it is N_{prev}) to N.

One native approach is to record the local clock of N_{prev} using the code running in N_{prev}'s enclave launched by SLAM, and subtract N's local time with this recorded time in N's SLAM enclave. This approach is problematic because if N or N_{prev} is faulty, either of the local times can be manipulated

by the faulty node(s), making the time subtracted from τ_d much less than the actual time spent in the transmission (i.e., τ_d is no longer conservative).

SLAM instead develops a trustworthy mechanism to record N's peers' half round-trip time cost conservatively as $N.HRTT$, and details are in §5.3. The scheduler module subtracts SLA transactions' τ_d with $N.HRTT$, and it ranks all the SLA transactions with smaller τ_d and appends all non-SLA transactions at the end of its priority queue. SLAM's multicast module also runs in SLAM's SGX enclave; it measures each SLA transaction's wait time in the scheduler module's queue as tx_N^{wait}, subtracts τ_d with this wait time, encrypts the transaction, and copies the encrypted transactions to the communication module outside the enclave. Since N itself can be faulty, its communication module can reorder or delay these transactions. Again, N's next hop node (suppose it is N_{next}) also uses its own $N_{next}.HRTT$ to update the transactions' remaining SLA deadlines conservatively.

4.2 Reliable Multicast

One key SLA requirement in SLAM is the sequential execution of per-client SLA transactions (i.e., gap-free), and the default P2P multicast protocols (typically Gossip and flooding) in existing blockchain systems are not designed to achieve the gap-filling task. Specifically, the flooding protocol [3] forwards all newly received transactions on each node to all the node's neighbors, an extremely bandwidth consuming process even for a low throughput blockchain [24]. Worse, flooding has no guarantee on filling gaps for smart contract transactions. In Gossip, when a node receives a transaction for the first time, rather than sending the transaction to all neighbors (flooding), it randomly selects a subset of neighbors, and forwards the transaction to them [20]. However, Gossip only provides weak guarantee on gap-filling (e.g., in Figure 1a, Gossip failed to meet 91% of SLA transactions within the same SLA deadline as SLAM's).

In order to provide high probability on the gap-filling task in a P2P network, three reliable multicast protocols are presented [10, 18, 25]. They typically follows a two-phase, disseminate-then-correct pattern, as shown in Figure 4 . The first phase is an unreliable, default Gossip that makes best-effort attempts to efficiently disseminate transactions. The second phase is an anti-entropy protocol, where each node contacts its peers to exchange both the lost and latest transactions. Erlay [25] is based on the bimodal reliable multicast [10] and integrated with Bitcoin [24]. Compared with Bitcoin's default flooding multicast, Erlay reduces the bandwidth consumption by 41%. In our evaluation, reliable multicast indeed can achieve better SLA guarantee than flooding and Gossip (Figure 1a). However, all these existing P2P reliable multicast are conceptually an uni-directional protocol: all nodes handle all the tasks of dissemination, gap-filling, and refreshing latest transactions. Therefore, each node needs to start from itself, uni-directionally and recursively inquiries their peers to infer the latest transactions throughout the network, a kind of flooding multicast [24]. After all, all existing reliable multicast protocols [**?**, 10, 18] except for Erlay are originally designed for MPI.

Our idea is that we can leverage the refreshing nature of a blockchain's consensus protocol to design a new bi-directional P2P reliable multicast protocol for SLAM, as shown in Figure 4. This protocol is simple, efficient, yet reliable on gap-filling. Specifically, on the forward direction, SLAM lets the P2P reliable multicast protocol conducts only gap-filling of lost SLA transactions among peers and to efficiently disseminate the complete sequences of per-client transactions throughout the network. On the backward direction, the consensus nodes of SLAM's blockchain disseminate committed blocks throughout the network; P2P nodes receiving the committed blocks safely skips the disseminations of the committed transactions from SLAM's scheduler, multicast, and communication modules. SLAM also safely skips the gap-filling task of lost transactions with smaller nonces than the committed transactions.

4.3 SLA Enforcement Probability Analysis

Now we analyze SLAM's probability in enforcing deadlines of SLA transactions. Recall SLAM's SLA variables (§3.2): given an uncongested P2P network and the median commit latency (life cycle) of an SLA transaction ($\tau_d + \tau_c$), suppose in every second, there are totally n SLA transactions generated from all clients, and suppose n is smaller than the consensus protocol's throughput. Now we investigate the probability p: the percentage of these SLA transactions that can meet their SLAs ($c \times T$), where c is 2 by default.

SLAM's reliable multicast protocol is based on Gossip [21], where transactions are propagated to the entire network in rounds. During the propagation rounds of a transaction, each independent node randomly selects a subset of its peers and forwards the transaction to the peers. This random transmission process on each node is a Poisson process [23]. Based on the fact that the superposition of independent Poisson processes is also a Poisson process [**?**], an SLA transaction's dissemination latency τ_d to the entire network satisfies the Poisson distribution.

The end-to-end commit latency of a transaction is $\tau_d + \tau_c$. Since we have an uncongested network, the consensus latency τ_c is small (29.0%) and relatively stable, confirmed in our evaluation. Therefore, $\tau_d + \tau_c$ also satisfies Poisson distribution. Suppose the median commit latency of a transaction is T (§3.3), according to Poisson Distribution's probability function $p(x,T) = \frac{e^{-T}T^x}{x!}$, if we set the SLA transaction's deadline as $2 \times T$, 96.0% of these transactions can meet their SLA. This high theoretical rate matches SLAM's actual SLA satisfaction rate in our evaluation.

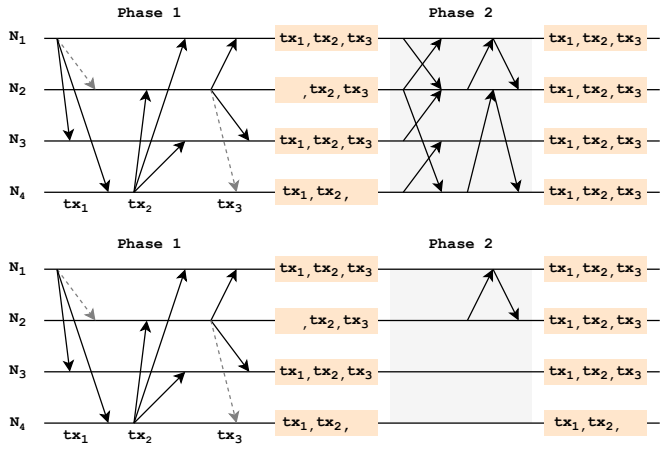


Figure 4: Message pattern. TODO bidirection

5 SLAM's Security Design and Analysis

This section presents our security guarantee: SLAM can maintain a high SLA satisfaction rate in §4.3 in face of attacks (e.g., selective targeted attacks mentioned in §3.3). As SLAM's SLA prioritization mechanism runs in each node's SGX, an attacker can only conduct attacks on messages exchanged between different nodes' enclaves. Specifically, an attacker can corrupt, reorder, drop, and delay messages. Since SLAM's protocol does not rely on message orders to enforce SLA, we do not need to maintain message orders. Since message drop and delay can not be distinguished in Internet, we call them deferring attacks and handle them together. The drop and delay attacks can be either selective or random. §5.1 shows how SLAM defends against message corruption attacks. §5.2 presents a message oblivious mechanism to handle selective drop and delay attacks. §5.3 presents a trustworthy (conservative) RTT mechanism to handle random drop and delay attacks.

5.1 Handling Message Corruption Attacks

The metadata in SLA transactions are sensitive. If faulty nodes maliciously access and modify the metadata, they can easily break SLAM's SLA guarantee. For example, faulty nodes can change the *deadline* of an SLA transaction from *5s* to *50s*, then the SLA transaction will not be prioritized even if it is stringent to disseminate, leading to SLA violation. To prevent such attack, SLAM uses SGX and protects SLA transactions' metadata updates on all SLAM nodes (§4.1). Specifically, we put all the uncommitted SLA transactions in enclaves and pack the scheduler logic as `ECalls` to securely read incoming transactions, and decide on which transactions should be disseminated. In this manner, even if a faulty node runs our scheduler module and tries to poison the metadata, she cannot access the metadata and control the disseminated messages because of the shield of SGX. We keep a small TCB of SLAM's trusted modules (§3.3), and we only put uncommitted SLA transactions into enclaves.

5.2 Handling Selective Deferring Attacks

In SLAM, faulty nodes can selectively defer P2P messages and easily violate the SLA of many clients' SLA transactions (e.g., the selective targeted attacks mentioned in §3.3). If SLAM's protocol messages in the network have different sizes (e.g., if `gapFill` is not larger than 10B while a transaction dissemination message size can reach 100B), this exposes a large attack interface to attackers. Thus, we design to make all P2P messages in SLAM oblivious by filling in extra dummy payload and encrypt messages with the same symmetric key. In this way, all messages in the SLAM network are oblivious (100B), and a faulty node can not distinguish whether a message is a dissemination message (either an SLA transaction or non-SLA transaction), a conservative HRTT ping-pong message (§5.3) or a `gapFill` message (§4.2).

5.3 Handling Random Deferring Attacks

However, even if SLAM's P2P messages are oblivious, it is still challenging to guarantee SLA requirements of transactions in an asynchronous network. Even if messages are oblivious, faulty nodes might randomly and arbitrarily drop/delay P2P messages. For instance, faulty nodes can defer the dissemination of transactions on some faulty nodes close to consensus nodes; if a naive SLA remaining deadline updating mechanism uses the differences of local system clocks of these faulty nodes to update the deadline of an SLA transaction (even if the updating code runs in SLAM's enclave), these faulty nodes can still actually greatly defer the transaction's one hop transmission and manipulate the nodes' local clocks to fool the transaction that it is far from being stringent (e.g., the transaction's one hop elapse time measured by the clocks between two faulty nodes is made negligible). Then, other transactions will have higher priorities to reach consensus nodes and make this fooled transaction be committed in several later blocks. This is a huge SLA violation, but the fooled transaction's SLA remaining deadline does not capture this attack (the deadline is no longer conservative).

To guarantee trustworthy (conservative) time value in SLAM's scheduler module and to reveal randomly deferred attacks, SLAM achieves a trustworthy per-node HRTT value with a Trustworthy HRTT maintaining protocol. Within regular time interval (10s, same as Ethereum's ping interval), the SLAM enclave on each node *N* (including a faulty node) encapsulates a `ping` message with the same format as normal transaction dissemination (around 100B) and sends that `ping` message with its latest HRTT value to all its peers. At

the time receiving a *ping* request, a peer node encapsulates a reply message within SLAM's local enclave and sends the reply message back to the inquiry node. The node N collects all the reply messages, all `ping` requests' HRTT values from all its peers, and selects the *highest* calculation result as the latest HRTT value.

This protocol provides trustworthy (conservative) HRTT value against the random deferring attacks. Since all faulty nodes need to forward transactions and update transactions' SLA metadata to avoid being kicked out from the network, faulty nodes have to keep sending messages including their enclaves sending HRTT messages. Also, a faulty node cannot manipulate an HRTT reply message because all messages are decrypted within SGX. Besides, our trustworthy Ping-Pong protocol can tolerate random deferring attacks and still make SLAM's SLA update mechanism conservatively. This is because we select the *highest* HRTT value, an inquiry node will get a statistical worst case of the actual network one hop delay (including the random delay conducted by faulty nodes, as faulty nodes cannot distinguish HRTT messages and transaction dissemination messages) and make SLA transactions more stringent (some transactions' SLA deadlines can be made negative, but still stringent and conservative), as the deadline for each SLA-transaction subtracts a statistically worst case value of HRTT.

In addition to use our Trustworthy Ping-Pong protocol to fetch trustworthy RTT, SLAM nodes can also get an SLA transaction tx's trusted local elapsed time on node N, denoted as tx_N^{wait} even if the local system clock is manipulated. This is trivial to achieve in the SGX domain, as tx_N^{wait} is accumulated by the multicast module's running process by counting CPU cycles, and the process never goes out of SLAM's enclave.

Overall, in SLAM's protocol, two critical variables used by the scheduler (§4), $N.HRTT$ and local elapsed time tx_N^{wait}, are both *conservative*: if SLAM nodes infer that an SLA transaction meets the SLA deadline, the transaction actually meets it with high probability, even though the transaction's dissemination has been deferred by some faulty nodes on some hops. Of course, if a client finds its committed SLA transactions with negative SLA remaining deadlines, these transactions may have actually met their SLA deadlines, because SLAM subtracts these two conservative variables from the deadlines.

6 Implementation

We implemented SLAM based on the latest Golang version of Ethereum [32] - a fully tested and actively maintained blockchain system on the Internet. We leveraged Ethereum's P2P library to build SLAM's reliable multicast component and we rewrote Ethereum's transaction logic for admitting, verifying and scheduling SLA transactions. We carefully selected sensitive functions and put these functions into SGX enclaves. Since SGX only provides C/C++ SDKs, we rewrote all sensitive functions in C and used cgo to invoke Ecalls. Totally, we modified 2037 lines of Golang code, and implemented the scheduler and reliable multicast component for 1705 lines of C code. For encryption/decryption, we used AES-256, a symmetric key library provided by the SGX SDK.

Config	Cluster	AWS Cloud
# Instance/Machine/Total	25/20/500	100/50/5,000
Block interval	5s	5s
Avg RTT	20ms	200ms
Deadline	16s	32s
pingInterval	10s	10s
Default consensus Protocol	PoA	PoA
Default workload	Online Payment	Online Payment
Bandwidth Limitation	20Mbps	30Mbps

Table 1: SLAM's default evaluation parameters

Joining of new nodes. When a node i wants to join SLAM, it needs to first find a member node j to do attestation. In a large scale deployment, finding a member node can be done via ab off-band peer discovery service (e.g., Ethereum's automatic peer discovery protocol). Specifically, node i joins SLAM with three steps. First, i launches its SLAM enclave. If i is newly joined node, it has to generate its account (pk_i,sk_i) and creates its monotonic counter. The node i then seals its account in storage in case network failures. If i is not new (e.g., recover from a previous state), it will unseal its account from the storage, increments and reads the counter value. Then node i sends a *join* request to the member node j for attestation. Second, node j does SGX remote attestation [22] that succeeds with a signed QUOTE Q_i from Intel IAS. Node i transfers it public key pk_i and counter value to node j' enclave through a secure channel created during attestation. Third, node j creates a registration transaction $\langle REG, pk_i, counter, addr_i, Q_i \rangle$ and broadcasts it to all j's peers. After the registration transaction is confirmed, node i successfully joins SLAM as a member node.

7 Evaluation

We evaluated SLAM's performance on both our own cluster and the AWS cloud [29], with other evaluation parameters shown in Table 1. In our cluster, each machine is equipped with 2.60GHZ Intel E3-1280 V6 CPU with SGX, 40Gbps NIC, 64GB memory and 1TB SSD. On the AWS cloud, we launched 50 m5d.24xlarge instances with 96 vCPUs, 4x900(SSD) and up to 25 Gigabit network bandwidth. We ran 100 SLAM nodes on each VM instance (5k nodes in total), with each SLAM node running in a docker container. To collect and analyze all SLAM member nodes' performance data, we also started one t2.xlarge instance with 4 vCPUs and moderate network bandwidth. All AWS instances are run in the same zone (Ohio). We run SGX in

cluster with hardware mode and in AWS with simulation mode because the AWS do not provide the hardware SGX mode. However, we show that SLAM's performance in simulation mode is roughly the same as hardware mode because SLAM's performance is bound to network latency.

We compared SLAM's performance with five p2p messaging protocols for blockchain systems, including three reliable multicast protocols (Erlay [25], Corrected protocol [18] and PODC protocol []), the basic Gossip protocol [21] and the Flooding protocol [3]. Among these baseline protocols, we evaluate the Flooding protocol by running origin Ethereum [32] (version eth/64) and is our code base; Gossip is a popular p2p messaging protocol that broadcast transactions to random peers in rounds, and is promising to disseminate a transaction to the whole network with controlled converge time; Among these reliable multicast protocols, Erlay is the only reliable multicast protocol that is really implemented on top of a blockchain system (i.e., Bitcoin [24]) and implemented a Bimodal correction protocol. Since Erlay is not open-sourced, we implemented Erlay in our own and integrated with Ethereum for evaluation; PODC protocol is implemented upon Bimodal protocol as well; Corrected protocol is the best-performed reliable multicast protocol to our best knowledge.

We run five different applications with different portion of SLA transactions, even more we simulated traffic spikes that could happen in the network. Our benchmark workload is *Online Payment*, which is the default application workload to evaluate SLAM's performance. We use *Online Payment* as the default workload because it has interactive transactions and is prevalent in the Internet. We set the transaction size as 100 bytes. The transaction size for baseline systems are either equal to or smaller than that of SLAM.

For all evaluated p2p messaging protocols with five application workload, we let a fraction of node generate transactions to simulate clients sending rate in order to reach peak throughput for each p2p messaging protocols. For SLA, we define SLA satisfaction rate as the percentage of SLA transactions that can meet their SLA requirements given by clients. We report throughput as txn/s by assuming all applications have the same transaction size as SLAM. We define the latency as the client persived time for all transactions.

In next part of evaluation, we focus on the following questions:

§7.1 How SLAM meets SLA for SLA transactions and what is the performance for all transactions?

§7.2 How resilent is the SLA and throughput on spike of SLA transactions, how does SLAM's reliable multicast protocol compared to other reliable multicast protocols?

§7.3 How resilent is the SLA and throughput to node failures in SLAM?

§7.4 How is the SLA and throughput for five applications in SLAM?

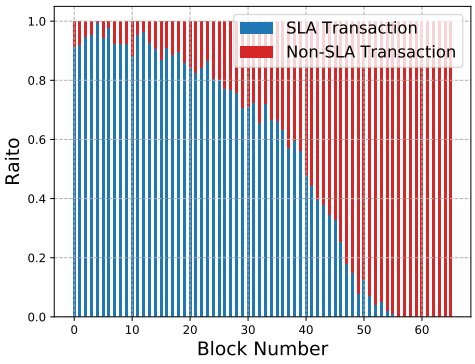


Figure 5: SLAM prioritizes SLA transactions.

§7.5 What is the limitations and potential future works of SLAM?

7.1 SLA and Performance

Figure 1a shows all systems' SLA satisfication rate for SLA transactions. And Figure 1b shows the Tput of all systems for all transactions. From 0 to 8s, the network generated 200 txs/s, and 20% of them were SLA transactions. This transaction rate matches the default PoA consensus protocol's throughput (about 200 txs/s). On 8s, we generated 200 additional SLA transactions for five seconds as a spike. Overall SLAM's SLA satisfication rate for SLA transactions dropped from 98.0% to 82.1% at about 15s and then quickly recover. The other systems' SLA satisfication rate all dropped significantly and recover much slower than SLAM. Gossip achieved the lowest SLA satisfication rate because it selects only a subset of peers to disseminate transactions without any gap-filling. This indicates that, SLAM achieves the highest SLA satisfication rate on the spike (all systems incur the same number of additional SLA transactions). Figure 1b shows that Gossip's throughput is the best, and SLAM incurred roughly 19.X% overhead on all transactions' throughput compared to Gossip. This is because SLAM's reliable multicast incurs a light-weight bi-directional gap-filling in addition to Gossip (Figure bidirectional). Other reliable multicast protocols and flooding incur much higher throughput overhead compared to Gossip.

To investigate the reasons of SLAM's high SLA satisfaction rate and its throughput overhead, we collected all systems' mirco-events from 0 to 8s in Table 3 and the same events from 8 to 40s in Table 4. On Table 3, SLAM's mean $\tau_d + \tau_c$ is much less than $2 * T = 16s$, which explains SLAM's high satisfaction rate of 98.X% before the spike comes at 8s. After 8s,SLAM incurs higher burden on disseminating SLA transactions, so its per transaction τ_d increases from 6s in Table 3 to 26.8s in Table 4, leading to a decreased SLA satisfaction rate. However, SLAM's SLA satisfaction rate was

Applications	SLA Txs	Non-SLA Txs	Traffic Spike
Mobile carriers [7, 12]	Pre-paid users (10%)	Post-paid users (90%)	×
Disease control [1, 2, 33]	High-risk events (50%)	Low-risk events (50%)	×
Onling payment [9, 27, 34]	Real time payments (90%)	Non-real time payments (10%)	✓
Voting [?, 6, 16]	Election management (10%)	Cast votes (90%)	×
CDN accounting [4, 17, 30]	Settlements (10%)	Usage data (90%)	✓

Table 2: SLAM's evaluated applications

still much better than the other three reliable multicast protocol because their τ_d increase by at least one order of magnitude, much larger than the SLA deadline $2 * T = 16s$. The reason SLAM's τ_d is much lower than all the other systems in Table 4 is two folds:(1) for reliable multicast protocols, SLAM incurred much fewer number of gap-filling invocations that spent much fewer time in gap-filling; (2) for traditional flooding and Gossip, they either incur high transaction miss rate on consensus nodes (64.1% for Gossip) or incur severe P2P network congestion (Flooding's τ_d is 123.7s).

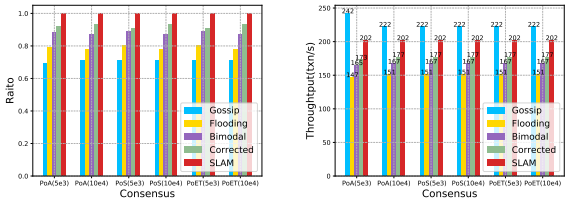
To understand the throughput of SLAM's SLA prioritize mechanism for SLA and non-SLA transactions, we broke down SLAM's portion of SLA transactions and non-SLA transactions in each committed block in Figure 5, which shows that SLAM's mechanism give high priority for SLA transactions in the first half of the committed blocks, and the non-SLA transactions took the majority in the second half. This implies that SLAM always schedules SLA transactions first (§4.2) and avoids non-SLA transactions to block SLA transactions' dissemination.

Regarding all systems' client persived latency on committed SLA transactions, Table 4 can give a good indication. By summing up the τ_d and τ_c columns SLAM achieved the lowest latency for SLA transactions among all systems since the spike occurred in the 8s. Note that this spike is fair for all systems, because the added 200 transactions were all SLA transactions. For non-SLA transactions, SLAM does tradeoff their commit latency, which are indicated in Figure 5: in this figure, a block is committed every 5s via the PoA consensus protocol, so the non-SLA transactions' mean commit latency in SLAM is at about the 36th block (180s). This actually matches SLAM's design goal. Overall, SLAM's mean commit latency for all transactions was lightly slower than Gossip (Table 4), which explains SLAM's modest throughput overhead for all transactions compared to Gossip's in Figure 1b.

Since there is no cloud provider that can run the SGX hardware on clouds, we ran SLAM's enclave code using Intel's SGX simulator on AWS. Table 5 shows the micro-events of running SLAM's enclave code on both the SGX simulator and running the same code on the SGX hardware of our cluster, which shows similar performance cost. SLAM's time spent in SGX is not the bottleneck of SLAM's performance, and we consider SLAM's performance results reported on AWS would be close to running physical SGX hardware on future clouds (if they can provide SGX hardware).

SGX	# ECalls/OCalls	Cost of Encryption and Decryption	Queueing time of SLA transactions at all hops without traffic spike	Queueing time of SLA transactions at all hops with traffic spike
Cluster	93	0.0186ms	3.4s	20.8s
AWS Cloud	115	0.023ms	4.8s	21.3s

Table 5: SLAM's SGX micro events through a transaction's lifetime



(a) SLA guarantees of different consensus protols in 5,000/10,000 scales

(b) Throughput of different consensus protols in 5,000/10,000 scales

Figure 6: Normalized performance of POA, POS and PoET in 5,000/10,000 scale

In addition to PoA, we also evaluated SLAM on different P2P network scales with two other high-throughput consensus protocols on Ethereum. The SLA transaction rate was the same as Figure **??** before As. Overall, SLAM achieves reasonable SLA satisfaction rate and reasonable overhead on throughput compare to Ethereum running with default Gossip multicast. Since typical applications (Table 2) are often deployed with 5000 P2P nodes [4], we consider SLAM is scalable to the network scale. Figure 1b also suggests that SLAM's throughput is mainly determined by consensus's throughput, due to SLAM's light weight gap-filling mechanism (§4.3)

7.2 Resilience on SLA Transaction Spikes

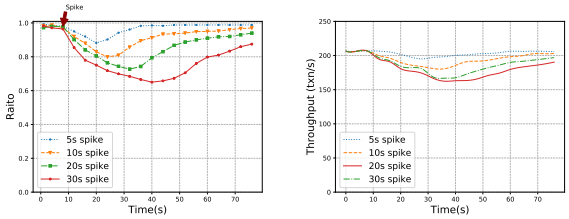
We also studied all systems' resilience to different degrees of SLA transaction spikes. We started with the same setting as the setting of 0s of Figure 7, and we varied the time length of the 200 additional SLA transactions from 10s, 20s

System	Consensus Latency (s) (τ_c)	Avg P2P Latency (s) (τ_d)	# Gap-filling invocation	Gap-filling Cost (s)	Miss rate of SLA transactions on validator
SLAM	5.98	6.04	5	1.03	3.01%
Flooding	6.24	8.7706	N/A	N/A	0.05%
Gossip	5.91	11.12	N/A	N/A	15%
Corrected	6.14	7.12	13	2.64	3.31%
Erlay	6.38	7.32	21	4.2	3.58%
PODC	6.27	7.77193	23	4.619	4.12%

Table 3: System breakdown without traffic spike in AWS (5000 nodes)

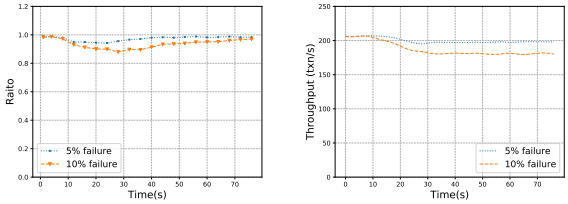
System	Consensus Latency (s) (τ_c)	Avg P2P Latency (s) (τ_d)	# Gap-filling invocation	Gap-filling Cost (s)	Miss rate of SLA transactions on validator
SLAM	7.53	26.82497	33	6.6	21.4%
Flooding	7.24	123.72	N/A	N/A	10.96%
Gossip	7.77	143.098	N/A	N/A	64.12%
Corrected	8.62	75.271	157	31.4	31.903%
Erlay	7.83	88.27318	198	39.81	35.91%
PODC	8.09	93.1	203	41.6129	36.171%

Table 4: System breakdown during traffic spike in AWS (5000 nodes)



(a) SLAM's SLA guarantee under transient network spike (b) SLAM's throughput under transient network spike

Figure 7: SLAM's SLA guarantee and performance under transient traffic spike



(a) SLAM's SLA guarantee under node failure (b) SLAM's throughput under node failure

Figure 8: SLAM's SLA guarantee and performance under node failure

to 30s. We compare SLAM's SLA transactions' satisfaction rate with the Corrected Gossip [18], the reliable multicast protocol that achieved the highest SLA satisfaction rate in Figure 1a. For the 10s-spike curve, SLAM's $n.RTT$ among all nodes at 37s (lowest SLA rate) is xxxms to xxxms. For the 30s-spike curve, SLAM's $n.RTT$ among all nodes at 37s (lowest SLA rate) is xxxms to xxxms. This indicates that SLAM's oblivious RTT update mechanism (§5.2) can properly capture packet delays. Figure 7 shows that, on different degrees of additional SLA transactions, SLAM' SLa satisfaction rate dropped to as low as 67.x% and then recovered. This is much better than Corrected Gossip even with just 5s of additional SLA transactions. Moreover, SLAM achieves much better throughput for all transactions than corrected Gossip, due to SLAM's lower number of gap-filling invocations and thus lower time cost spent on gap-filling (see Table spike).

7.3 Robustness on Node Failures

Node failures in a P2P network can be triggered by hardware failures or DoS attacks, and such failures are more severe than traffic spikes. We measured SLAM's SLA satisfaction rate and throughput with node failures, as shown in Figure 8. On 8s, we randomly selected 10% of nodes in SLAM's network and kill them. SLAM's SLA satisfaction rate dropped from 98.X% to about 89.X%, and its throughput for all transactions dropped from about 201 txs/s to 150 txs/s. In fact, given a looser P2P network topology, SLAM's SLA prioritization mechanism has to reconnect peers and recompute the conservative RTTs (§5.3) for nodes, and new peers in this killing setting are often farer, leading to more stringent SLA deadline during the dissemination.

7.4 SLA and Performance on Applications

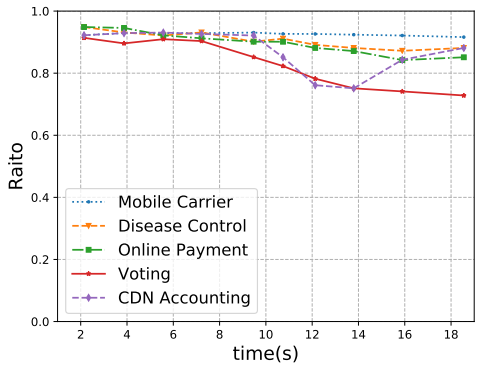


Figure 9: SLA guarantee under running with five different applications

The above evaluation focus on evaluating the online payment application with different systems on different scenarios. We deployed five applications (Table 2) in SLAM individually and measured their SLA and throughput according to their own application settings (e.g., portion of SLA transactions and spikes). Figure 9 shows the results for these applications, this suggests that SLAM's SLA mechanisms (§4 and §5) are generic for diverse applications with different settings.

7.5 Discussion

Faulty nodes maliciously tamper with the SLA metadata (remaining deadline).

SLAM has two limitations. First, SLAM's transaction scheduling mechanism is based on Intel SGX. It is essential to protect the SLA metadata in an untrusted environment with faulty nodes and to conservatively schedule transactions when traffic spikes and node failures occur. Nowadays, SGX is prevalent in modern CPUs. More and more blockchain consensus protocols are developed with SGX.

Second, SLAM's performance are designed to favor SLA transactions and aggravate non-SLA transactions. Figure 5 indicates that SLAM highly favors SLA transactions and defers non-SLA transactions. After all, more and more Internet-wide blockchain applications are developed with the stateful smart contracts, and some transactions are submitted in an interactive way in browsers or mobile Apps. These applications generally desire more stringent SLA guarantee for these transactions, so this trade-off is worthwhile.

SLAM, the first SLA dissemination system for permissioned blockchains, enables the research community to develop even more exciting blockchain systems and applications with diverse SLAs requirements. For instance, SLAM can facilitate the development of new heterogeneous blockchain applications consisting of fine-grained SLA requirements (e.g., a blockchain application consisting of online payment, auction, and clearing transactions). In addition, because SLAM's SLA scheduling mechanism is conservative (§4.2), SLAM can make SLA guided verifications feasible (e.g., trustworthy SLA incentive and auditing mechanisms) and can match the demands of blockchain-driven CDN networks [4] and auditing applications [26]. Last but not least, SLAM may also be used in permissionless blockchains [24], because all permissionless blockchains prioritize transactions base on transaction fees without any SLA guarantee. We leave these exciting applications and innovations for future works.

8 Related Work

blockchain applications permissioned blockchain Applications on blockchain smart contract deadline online interactive

reliable mulcicast authorgonal bi-directional Erlay

P2P and SLA DoS crucial make SLA enforcement very hard Erlay

9 Conclusion

We have present SLAM, the first permissioned blockchain system that can meets diverse SLAs of different applications. SLAM's integration with Ethereum has the potential to attract broad Internet-wide applications with SLA requirements to be developed upon. This will not only greatly improve the reliability of these applications, but also greatly improving the efficiency of both user perceived latency and network bandwidth.

References

[1] CDC is Testing Blockchain to Monitor the Country's Health in Real Time. https://www.nextgov.com/emerging-tech/2018/11/cdc-testing-blockchain-monitor-countrys-health-real-t 152622/, 2018.

[2] How IBM and the CDC are testing blockchain to track health issues like the opioid crisis. https://www.fastcompany.com/90231255/how-ibm-and-the-cdc-are-testing-blockchain-to-track-h 2018.

[3] Karl Aberer, Philippe Cudré-Mauroux, Anwitaman Datta, Zoran Despotovic, Manfred Hauswirth, Magdalena Punceva, and Roman Schmidt. P-grid: a self-organizing structured p2p system. *ACM SiGMOD Record*, 32(3):29–33, 2003.

[4] Elif Ak and Berk Canberk. Bcdn: A proof of concept model for blockchain-aided cdn orchestration and routing. *Computer Networks*, 161:162–171, 2019.

[5] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, et al. Hyperledger fabric: a distributed operating system for permissioned blockchains. In *Proceedings of the Thirteenth EuroSys Conference*, pages 1–15, 2018.

[6] Ahmed Ben Ayed. A conceptual secure blockchain-based electronic voting system. *International Journal of Network Security & Its Applications*, 9(3):01–09, 2017.

[7] Jere Backman, Seppo Yrjölä, Kristiina Valtanen, and Olli Mämmelä. Blockchain network slice broker in 5g: Slice leasing in factory of the future use case. In *2017 Internet of Things Business Models, Users, and Networks*, pages 1–8. IEEE, 2017.

[8] Arati Baliga, I Subhod, Pandurang Kamat, and Siddhartha Chatterjee. Performance evaluation of the quorum blockchain platform. *arXiv preprint arXiv:1809.03421*, 2018.

[9] Mathieu Baudet, Avery Ching, Andrey Chursin, George Danezis, François Garillot, Zekun Li, Dahlia Malkhi, Oded Naor, Dmitri Perelman, and Alberto Sonnino. State machine replication in the libra blockchain, 2019.

[10] Kenneth P Birman, Mark Hayden, Oznur Ozkasap, Zhen Xiao, Mihai Budiu, and Yaron Minsky. Bimodal multicast. *ACM Transactions on Computer Systems (TOCS)*, 17(2):41–88, 1999.

[11] Vitalik Buterin et al. A next-generation smart contract and decentralized application platform. *white paper*, 3(37), 2014.

[12] Abdulla Chaer, Khaled Salah, Claudio Lima, Pratha Pratim Ray, and Tarek Sheltami. Blockchain for 5g: opportunities and challenges. In *2019 IEEE Globecom Workshops (GC Wkshps)*, pages 1–6. IEEE, 2019.

[13] Stephen Checkoway and Hovav Shacham. Iago attacks: why the system call api is a bad untrusted rpc interface. *ACM SIGARCH Computer Architecture News*, 41(1):253–264, 2013.

[14] Message Passing Interface Forum. Mpi: A message-passing interface standard version 2.2, September 2009.

[15] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th Symposium on Operating Systems Principles*, pages 51–68, 2017.

[16] Rifa Hanifatunnisa and Budi Rahardjo. Blockchain based e-voting recording system design. In *2017 11th International Conference on Telecommunication Systems Services and Applications (TSSA)*, pages 1–6. IEEE, 2017.

[17] Nicolas Herbaut and Nicolas Negru. A model for collaborative blockchain-based video delivery relying on advanced network services chains. *IEEE Communications Magazine*, 55(9):70–76, 2017.

[18] Torsten Hoefler, Amnon Barak, Amnon Shiloh, and Zvi Drezner. Corrected gossip algorithms for fast reliable broadcast on unreliable systems. In *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 357–366. IEEE, 2017.

[19] David Lazar, Yossi Gilad, and Nickolai Zeldovich. Karaoke: Distributed private messaging immune to passive traffic analysis. In *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*, pages 711–725, 2018.

[20] Joao Leitao, José Pereira, and Luis Rodrigues. Hyparview: A membership protocol for reliable gossip-based broadcast. In *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*, pages 419–429. IEEE, 2007.

[21] Joao Leitao, José Pereira, and LuÍs Rodrigues. Gossip-based broadcast. In *Handbook of Peer-to-Peer Networking*, pages 831–860. Springer, 2010.

[22] Frank McKeen, Ilya Alexandrovich, Alex Berenzon, Carlos V Rozas, Hisham Shafi, Vedvyas Shanbhogue, and Uday R Savagaonkar. Innovative instructions and software model for isolated execution. *HASP @ ISCA*, 10, 2013.

[23] Yves Mocquard, Bruno Sericola, and Emmanuelle Anceaume. Probabilistic analysis of rumor-spreading time. *INFORMS Journal on Computing*, 2019.

[24] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. Technical report, Manubot, 2019.

[25] Gleb Naumenko, Gregory Maxwell, Pieter Wuille, Alexandra Fedorova, and Ivan Beschastnikh. Erlay: Efficient transaction relay for bitcoin. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 817–831, 2019.

[26] Bruce Schneier and John M Kelsey. Event auditing system, November 2 1999. US Patent 5,978,475.

[27] John Taskinsoy. This time is different: Facebook's libra can improve both financial inclusion and global financial stability as a viable alternative currency to the us dollar. *Available at SSRN 3434493*, 2019.

[28] Nirvan Tyagi, Yossi Gilad, Derek Leung, Matei Zaharia, and Nickolai Zeldovich. Stadium: A distributed metadata-private messaging system. In *Proceedings of the 26th Symposium on Operating Systems Principles*, pages 423–440, 2017.

[29] Jinesh Varia. Migrating your existing applications to the aws cloud. *A Phase-driven Approach to Cloud Migration*, 2010.

[30] Thang X Vu, Symeon Chatzinotas, and Björn Ottersten. Blockchain-based content delivery networks: Content transparency meets user privacy. In *2019 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 1–6. IEEE, 2019.

[31] Wenhao Wang, Guoxing Chen, Xiaorui Pan, Yinqian Zhang, XiaoFeng Wang, Vincent Bindschaedler, Haixu Tang, and Carl A Gunter. Leaky cauldron on the dark land: Understanding memory side-channel hazards in sgx. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2421–2434, 2017.

[32] Gavin Wood et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.

[33] Xiao Yue, Huiju Wang, Dawei Jin, Mingqiang Li, and Wei Jiang. Healthcare data gateways: found healthcare intelligence on blockchain with novel privacy risk control. *Journal of medical systems*, 40(10):218, 2016.

[34] Yinghui Zhang, Robert Deng, Ximeng Liu, and Dong Zheng. Outsourcing service fair payment based on blockchain and its applications in cloud computing. *IEEE Transactions on Services Computing*, 2018.